FINAL Unit3 FRQ - Topic order

1. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves methods that distribute text across lines of an electronic sign. The electronic sign and the text to be displayed on it are represented by the `Sign` class. You will write the complete `Sign` class, which contains a constructor and two methods.

The `Sign` class constructor has two parameters. The first parameter is a `String` that contains the message to be displayed on the sign. The second parameter is an `int` that contains the *width* of each line of the sign. The width is the positive maximum number of characters that can be displayed on a single line of the sign.

A sign contains as many lines as are necessary to display the entire message. The message is split among the lines of the sign without regard to spaces or punctuation. Only the last line of the sign may contain fewer characters than the width indicated by the constructor parameter.

The following are examples of a message displayed on signs of different widths. Assume that in each example, the sign is declared with the width specified in the first column of the table and with the message `"Everything on sale, please come in"`, which contains 34 characters.

Width of the Sign	Sign Display

15	`Everything on s`
	`ale, please com`
	`e in`

17	`Everything on sal` `e, please come in`
40	`Everything on sale, please come in`

In addition to the constructor, the `Sign` class contains two methods.

The `numberOfLines` method returns an `int` representing the number of lines needed to display the text on the sign. In the previous examples, `numberOfLines` would return `3`, `2`, and `1`, respectively, for the sign widths shown in the table.

The `getLines` method returns a `String` containing the message broken into lines separated by semicolons `(;)` or returns `null` if the message is the empty string. The constructor parameter that contains the message to be displayed will not include any semicolons. As an example, in the first row of the preceding table, `getLines` would return `"Everything on s;ale, please com;e in"`. No semicolon should appear at the end of the `String` returned by `getLines`.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `Sign`.

Statement	Method Call Return Value (blank if none)	Explanation
`String str;`		
`int x;`		

FINAL Unit3 FRQ - Topic order

Statement	Method Call Return Value (blank if none)	Explanation
`Sign sign1 = new Sign("ABC222DE", 3);`		The message for `sign1` contains 8 characters, and the sign has lines of width 3.
`x = sign1.numberOfLines();`	`3`	The sign needs three lines to display the 8-character message on a sign with lines of width 3.
`str = sign1.getLines();`	`"ABC;222;DE"`	Semicolons separate the text displayed on the first, second, and third lines of the sign.
`str = sign1.getLines();`	`"ABC;222;DE"`	Successive calls to `getLines` return the same value.
`Sign sign2 = new Sign("ABCD", 10);`		The message for `sign2` contains 4 characters, and the sign has lines of width 10.
`x = sign2.numberOfLines();`	`1`	The sign needs one line to display the 4-character message on a sign with lines of width 10.
`str = sign2.getLines();`	`"ABCD"`	No semicolon appears, since the text to be displayed fits on the first line of the sign.
`Sign sign3 = new Sign("ABCDEF", 6);`		The message for `sign3` contains 6 characters, and the sign has lines of width 6.
`x = sign3.numberOfLines();`	`1`	The sign needs one line to display the 6-character message on a sign with lines of width 6.
`str = sign3.getLines();`	`"ABCDEF"`	No semicolon appears, since the text to be displayed fits on the first line of the sign.
`Sign sign4 = new Sign("", 4);`		The message for `sign4` is an empty string.
`x = sign4.numberOfLines();`	`0`	There is no text to display.
`str = sign4.getLines();`	`null`	There is no text to display.
`Sign sign5 = new Sign("AB_CD_EF", 2);`		The message for `sign5` contains 8 characters, and the sign has lines of width 2.
`x = sign5.numberOfLines();`	`4`	The sign needs four lines to display the 8-character message on a sign with lines of width 2.
`str = sign5.getLines();`	`"AB;_C;D_;EF"`	Semicolons separate the text displayed on the four lines of the sign.

Write the complete `Sign` class. Your implementation must meet all specifications and conform to the examples shown in the preceding table.

2. The Car class, which you will write, is used to represent information about a car.

Car objects are created by calls to a constructor with two double parameters. The first parameter represents the number of gallons of fuel in the car. The second parameter represents the average number of miles the car can travel per gallon of fuel.

The Car class contains a canMakeTrip method, which determines whether there is enough fuel in the car to make a trip. This method has a double parameter that represents the distance, in miles, of a potential trip. If there is enough fuel, the

FINAL Unit3 FRQ - Topic order

method updates the current amount of fuel in the car and returns true. If there is not enough fuel for the entire distance of the trip, the amount of fuel is unchanged and the method returns false.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than Car.

Statement	Return Value (blank if no value)	Explanation
boolean possible;		
Car carOne = new Car(10.0, 30.0);		A Car object named CarOne is constructed with 10 gallons of fuel and an average of 30 miles per gallon.
possible = carOne.canMakeTrip(120.0);	true	At 30 miles per gallon, a 120-mile trip will require 4 gallons of fuel. Since this is less than or equal to the 10 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 6 gallons.
possible = carOne.canMakeTrip(90.0);	true	At 30 miles per gallon, a 90-mile trip will require 3 gallons of fuel. Since this is less than or equal to the 6 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 3 gallons.
possible = carOne.canMakeTrip(225.0);	false	At 30 miles per gallon, a 225-mile trip will require 7.5 gallons of fuel. Since this is more than the 3 gallons of fuel remaining, the trip cannot be made and the amount of fuel is unchanged.
Car carTwo = new Car(12.0, 15.0);		A Car object named carTwo is constructed with 12 gallons of fuel and an average of 15 miles per gallon.
possible = carTwo.canMakeTrip(300.0);	false	At 15 miles per gallon, a 300-mile trip will require 20 gallons of fuel. Since this is more than the 12 gallons of fuel remaining, the trip cannot be made and the amount of fuel is unchanged.
possible = carTwo.canMakeTrip(180.0);	true	At 15 miles per gallon, a 180-mile trip will require 12 gallons of fuel. Since this is less than or equal to the 12 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 0 gallons.

Write the complete `Car` class. Your implementation must meet all specifications and conform to the examples shown in the table.

3. The Cube class represents cubes that are used in a board game. There are three types of cubes.

- Number cubes, whose faces are labeled with the numbers 1 through 6

FINAL Unit3 FRQ - Topic order

- Letter cubes, whose faces are labeled with the letters A through F
- Color cubes, whose faces are labeled with the colors red, orange, yellow, green, blue, and purple

A partial Cube class definition is shown.

```
public class Cube
{
  /**
   *  Constructs a Cube object. If the parameter is
   *  equal to "number", the cube will be a number
   *  cube. If the parameter is equal to "letter", the
   *  cube will be a letter cube. If the parameter is
   *  equal to "color", the cube will be a color cube.
   *  Precondition: cubeType is equal to "number",
   *                "letter", or "color".
   */
  public Cube(String cubeType)
  { /* implementation not shown */ }

  /**
   *  Returns "1", "2", "3", "4", "5", or "6" if
   *  the cube is a number cube, returns "A", "B",
   *  "C", "D", "E", or "F" if the cube is a
   *  letter cube, and returns "red", "orange",
   *  "yellow", "green", "blue", or "purple" if
   *  the cube is a color cube
   */
  public String roll()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

You will write a class CubePair to represent two Cube objects that are used in a game. The CubePair class should have a one-parameter constructor, a two-parameter constructor, and a rollCubes method.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than Cube or CubePair.

Statement	Possible Return Value (blank if no value)	Explanation
CubePair p1 = new CubePair("number", "color");		Constructs a CubePair object with two Cube attributes: one number cube and one color cube.

FINAL Unit3 FRQ - Topic order

Statement	Possible Return Value (blank if no value)	Explanation
p1.rollCubes();	"2yellow"	Returns a String consisting of the results obtained by rolling the two cubes associated with p1.
CubePair p2 = new CubePair("number");		Constructs a CubePair object with two Cube attributes that are both number cubes.
p2.rollCubes();	"53"	Returns a String consisting of the results obtained by rolling the two cubes associated with p2.
CubePair p3 = new CubePair("letter");		Constructs a CubePair object with two Cube attributes that are both letter cubes.
p3.rollCubes();	"DC"	Returns a String consisting of the results obtained by rolling the two cubes associated with p3.

Assume there is a precondition that all arguments passed to a CubePair constructor are equal to "number", "letter", or "color".

Write the complete CubePair class. Your implementation must meet all specifications and conform to the examples shown in the table.

4. The CupcakeMachine class, which you will write, represents a cupcake vending machine, an automated machine that dispenses cupcakes. CupcakeMachine objects are created by calls to a constructor with two parameters.

- The first parameter is an int that represents the number of cupcakes that the vending machine has been stocked with. Assume that this value will be greater than or equal to 0.
- The second parameter is a double that represents the cost, in dollars, per cupcake. Assume that this value will be greater than 0.0.

The CupcakeMachine class contains a takeOrder method, which determines whether a cupcake order can be filled. A cupcake order can be filled if there are at least as many cupcakes in the vending machine as there are in the order.

A cupcake order is represented by a single int parameter to the takeOrder method. Assume that all values passed to the takeOrder method are positive.

If an order can be filled, the method updates the number of cupcakes remaining in the machine and returns a String containing information about the order. The returned string indicates the order number and the cost of the order, as shown in the following table. Order numbers begin at 1 and increase by 1 for each order filled.

If the order cannot be filled because the vending machine does not have enough cupcakes, the takeOrder method should return the message "Order cannot be filled". In this case, the number of cupcakes available in the machine is unchanged and no order number is given to the order.

The following table contains a sample code execution sequence and the corresponding results. The code execution

sequence appears in a class other than CupcakeMachine.

Statement	Return Value (blank if no value)	Explanation
String info;		
CupcakeMachine c1 = new CupcakeMachine(10, 1.75);		CupcakeMachine c1 is constructed with 10 cupcakes and a cost of $1.75 per cupcake.
info = c1.takeOrder(2);	"Order number 1, cost $3.5"	A customer orders 2 cupcakes for a total cost of 2 × $1.75. There are now 8 cupcakes remaining in the machine.
info = c1.takeOrder(3);	"Order number 2, cost $5.25"	A customer orders 3 cupcakes for a total cost of 3 × $1.75. There are now 5 cupcakes remaining in the machine.
info = c1.takeOrder(10);	"Order cannot be filled"	A customer attempts to order 10 cupcakes. The machine only has 5 cupcakes available, so no order is made.
info = c1.takeOrder(1);	"Order number 3, cost $1.75"	A customer orders 1 cupcake for a total cost of $1.75. There are now 4 cupcakes remaining in the machine.
CupcakeMachine c2 = new CupcakeMachine(10, 1.5);		CupcakeMachine c2 is constructed with 10 cupcakes and a cost of $1.50 per cupcake.
info = c2.takeOrder(10);	"Order number 1, cost $15.0"	A customer orders 10 cupcakes for a total cost of 10 × $1.50. There are now 0 cupcakes remaining in the machine.

Write the complete CupcakeMachine class. Your implementation must meet all specifications and conform to the examples shown in the table.

5. The LapTracker class, which you will write, is is used to count the number of laps a person walks around a track.

LapTracker objects are created by calls to a constructor with an int parameter that indicates how often to reset the LapTracker, with a precondition that the parameter is always positive. Newly constructed LapTracker objects start with a lap count of 0.

The LapTracker class contains an addLaps method, which adds laps to the lap count. This method has an int parameter that represents the number of laps to be added, with a precondition that the parameter is always positive. A LapTracker object repeatedly resets its lap count to 0 after a number of calls to addLaps, as indicated by its constructor parameter. The addLaps method returns an int that represents the lap count, as illustrated in the following table.

FINAL Unit3 FRQ - Topic order

The table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than LapTracker.

Statement	Return Value (blank if no value)	Explanation
int count;		
LapTracker amy = new LapTracker(3);		A LapTracker object named amy is constructed with an initial lap count of 0. The lap count will reset to 0 after every third call to amy's addLaps method.
count = amy.addLaps(8);	8	8 laps are added to the total lap count for amy.
count = amy.addLaps(12);	20	12 laps are added to the total lap count for amy.
count = amy.addLaps(10);	30	10 laps are added to the total lap count for amy. The addLaps method has been called on amy three times, so the next time the lap count will restart from 0.
count = amy.addLaps(11);	11	After the reset, 11 laps are added to the total lap count for amy.
count = amy.addLaps(5);	16	5 laps are added to the total lap count for amy.
count = amy.addLaps(13);	29	13 laps are added to the total lap count for amy. The addLaps method has been called on amy three more times, so the next time the lap count will restart from 0.
count = amy.addLaps(7);	7	After the reset, 7 laps are added to the total lap count for amy.
LapTracker eve = new LapTracker(2);		A LapTracker object named eve is constructed with an initial lap count of 0. The lap count will reset to 0 after every second call to eve's addLaps method.
count = eve.addLaps(20);	20	20 laps are added to the total lap count for eve.
count = eve.addLaps(13);	33	13 laps are added to the total lap count for eve. The addLaps method has been called on eve two times, so the next time the lap count will restart from 0.

FINAL Unit3 FRQ - Topic order

Statement	Return Value (blank if no value)	Explanation
count = eve.addLaps(10);	10	After the reset, 10 laps are added to the total lap count for eve.

Write the complete LapTracker class. Your implementation must meet all specifications and conform to the examples shown in the table.

6. The TrashCan class, which you will write, represents a trash can.

TrashCan objects are created by calls to a constructor with a double parameter that represents the trash can's capacity, which is the maximum weight, in pounds, that a TrashCan object can hold. Assume that this value will be greater than or equal to 0. When a Trashcan object is constructed, it is initially empty and its contents have a weight of 0 pounds.

The TrashCan class contains an acceptTrash method, which has a double parameter that represents the weight, in pounds, of the trash that will be deposited into the trash can. The amount to be deposited will always be greater than 0 and less than the capacity of the can. If there is already trash in the trash can, the amount to be deposited, combined with the existing trash, may reach or exceed the can's capacity. In this case, the trash can will be filled to its capacity and then emptied before the remaining trash is accepted. The trash can should always be emptied when the weight of the trash in the can reaches the can's capacity. The acceptTrash method returns a double that represents the number of pounds of trash that can still be added to the trash can before it is completely filled.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than TrashCan.

Statement	Return Value (blank if no value)	Explanation
double remaining;		
TrashCan kitchen = new TrashCan(10.0);		A TrashCan object named kitchen is constructed with a capacity of 10 pounds of trash. Initially there are 0 pounds of trash in the trash can.
remaining = kitchen.acceptTrash(2.5);	7.5	kitchen now holds 2.5 pounds of trash and can take 7.5 more pounds before it needs to be emptied.
remaining = kitchen.acceptTrash(3.5);	4.0	kitchen now holds 6 pounds of trash and can take 4 more pounds before it needs to be emptied.

FINAL Unit3 FRQ - Topic order

Statement	Return Value (blank if no value)	Explanation
remaining = kitchen.acceptTrash(6.0);	8.0	kitchen accepts 4 pounds of trash, reaching the 10-pound capacity, and is then emptied. The remaining 2 pounds of trash are then accepted. kitchen now holds 2 pounds of trash and can take 8 more pounds before it needs to be emptied.
TrashCan bedroom = new TrashCan(3.0);		A TrashCan object named bedroom is constructed with a capacity of 3 pounds of trash. Initially there are 0 pounds of trash in the trash can.
remaining = bedroom.acceptTrash(1.0);	2.0	bedroom now holds 1 pound of trash and can take 2 more pounds before it needs to be emptied.
remaining = bedroom.acceptTrash(3.2);	1.8	bedroom accepts 2 pounds of trash to reach the 3-pound capacity and then is emptied. The remaining 1.2 pounds of trash are then accepted. bedroom now holds 1.2 pounds of trash and can take 1.8 more pounds before it needs to be emptied.

Write the complete TrashCan class. Your implementation must meet all specifications and conform to the examples shown in the table.

7. The Apple class is used to store information about the apples to be used to bake apple pies. A partial declaration of the Apple class is shown.

```
public class Apple
{
  /**
   * Returns the weight of the apple, in pounds
   */
  public double getWeight()
  { /* implementation not shown */ }

  /**
   * Returns true if the apple is rotten and returns false otherwise
   */
  public boolean isRotten()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The AppleCrate class maintains a two-dimensional array of apples. A partial declaration of the AppleCrate class is shown.

FINAL Unit3 FRQ - Topic order

```
public class AppleCrate
{
 private Apple[][] apples;

  /**
   * Returns the number of pies that can be made from the 2D array apples
   * if each pie requires poundsPerPie pounds of apples. If an apple is
   * rotten, it cannot be used in a pie, nor can any adjacent apple.
   * Preconditions: apples has at least two rows and at least two columns.
   *              No elements in apples are null.
   */
  public int numberOfPies(double poundsPerPie)
  { /* to be implemented */}

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

Write the AppleCrate method numberOfPies. The method should return the total number of pies that can be made from the apples in the two-dimensional array apples if each pie requires poundsPerPie pounds of apples.

Rotten apples cannot be used in a pie, nor can any apples adjacent to a rotten apple. The apples adjacent to a given apple are the apples one element above, one element below, one element to the left of, and one element to the right of the given apple.

Suppose apples has the following contents. For each apple, the first value represents the apple's weight, in pounds, and the second value indicates whether the apple is rotten.

0.3 false	0.4 false	0.35 false	0.5 false	0.45 false	0.4 true
0.42 false	0.38 false	0.33 false	0.4 false	0.41 false	0.44 false
0.32 false	0.44 false	0.35 true	0.49 false	0.29 false	0.32 false
0.3 false	0.4 false	0.35 false	0.5 false	0.45 false	0.4 false

The shaded elements cannot be used in a pie because they represent rotten apples or apples that are adjacent to rotten apples.

For the contents of apples shown, numberOfPies(2.0) should return 3. The total weight of the apples available for pies (the non-shaded elements) is 6.14 pounds. If 2.0 pounds of apples are needed for each pie, 3 whole pies can be made, since $6.14/2.0 = 3.07$.

Complete method numberOfPies.

FINAL Unit3 FRQ - Topic order

```
/**
 * Returns the number of pies that can be made from the 2D array apples
 * if each pie requires poundsPerPie pounds of apples. If an apple is
 * rotten, it cannot be used in a pie, nor can any adjacent apple.
 * Preconditions: apples has at least two rows and at least two columns.
 *              No elements in apples are null.
 */
public int numberOfPies(double poundsPerPie)
```

8. The Appointment class is used to store information about a person's scheduled appointments. A partial declaration of the Appointment class is shown.

```
public class Appointment
{
  /**
   * Returns the status of the appointment ("free", "busy", etc.)
   */
  public String getStatus()
  { /* implementation not shown */ }

  /**
   * Returns the room number of the appointment location
   */
  public int getRoomNumber()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

The Schedule class maintains a two-dimensional array of appointments that represents a person's schedule. A partial declaration of the Schedule class is shown.

```
public class Schedule
{
  private Appointment[][] sched;

  /**
   * Returns the index of a column containing the fewest occurrences
   * of the status indicated by the parameter target
   * Preconditions: sched is not null and no elements of sched are null.
   *              sched has at least one row and at least one column.
   */
  public int columnWithFewest(String target)
  { /* to be implemented */}

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

FINAL Unit3 FRQ - Topic order

When an element of the two-dimensional array sched is accessed, the first index is used to specify the row and the second index is used to specify the column.

Write the Schedule method columnWithFewest. The method should return the index of a column in sched that contains the fewest occurrences of the parameter target. If there are multiple columns that have the fewest number of occurrences of target, any of their column indices can be returned.

Suppose sched has the following contents. For each element, the first value is the appointment status and the second value is the room number.

	0	1	2	3	4
0	"free" 100	"free" 100	"free" 100	"busy" 206	"busy" 204
1	"free" 100	"free" 100	"busy" 304	"busy" 206	"busy" 202
2	"hold" 201	"busy" 105	"busy" 205	"free" 100	"busy" 205
3	"busy" 204	"free" 100	"busy" 310	"hold" 110	"free" 100
4	"busy" 204	"hold" 201	"hold" 310	"busy" 105	"free" 100
5	"busy" 105	"busy" 208	"hold" 310	"busy" 105	"free" 100

For these contents of sched, the expected behavior of columnWithFewest is as follows.

- The call columnWithFewest("busy") should return 1 because "busy" appears two times in column 1 and more than two times in each of the other columns.
- The call columnWithFewest("free") should return either 2 or 3 because "free" appears one time in column 2, one time in column 3, and more than one time in each of the other columns.
- The call columnWithFewest("hold") should return 4 because "hold" appears zero times in column 4 and one or more times in each of the other columns.

Complete method columnWithFewest.

```
/**
 * Returns the index of a column containing the fewest occurrences
 * of the status indicated by the parameter target
 * Preconditions: sched is not null and no elements of sched are null.
 *                sched has at least one row and at least one column.
 */
public int columnWithFewest(String target)
```

9. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

FINAL Unit3 FRQ - Topic order

This question involves a scoreboard for a game. The game is played between two teams who alternate turns so that at any given time, one team is active and the other team is inactive. During a turn, a team makes one or more plays. Each play can score one or more points and the team's turn continues, or the play can fail, in which case no points are scored and the team's turn ends. The `Scoreboard` class, which you will write, is used to keep track of the score in a game.

The `Scoreboard` class contains a constructor and two methods.

The constructor has two parameters. The first parameter is a `String` containing the name of team 1, and the second parameter is a `String` containing the name of team 2. The game always begins with team 1 as the active team.

The `recordPlay` method has a single nonnegative integer parameter that is equal to the number of points scored on a play or `0` if the play failed. If the play results in one or more points scored, the active team's score is updated and that team remains active. If the value of the parameter is `0`, the active team's turn ends and the inactive team becomes the active team. The `recordPlay` method does not return a value.

The `getScore` method has no parameters. The method returns a `String` containing information about the current state of the game. The returned string begins with the score of team 1, followed by a hyphen (`"-"`), followed by the score of team 2, followed by a hyphen, followed by the name of the team that is currently active.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `Scoreboard`.

Statement	Value Returned (blank if none)	Explanation
`String info;`		
`Scoreboard game = new Scoreboard("Red", "Blue");`		`game` is a new `Scoreboard` for a game played between team 1, whose name is`"Red"`, and team 2, whose name is `"Blue"`. The active team is set to team 1.
`info = game.getScore();`	`"0-0-Red"`	
`game.recordPlay(1);`		Team 1 earns 1 point because the game always begins with team 1 as the active team.
`info = game.getScore();`	`"1-0-Red"`	
`game.recordPlay(0);`		Team 1's play failed, so team 2 is now active.
`info = game.getScore();`	`"1-0-Blue"`	
`info = game.getScore();`	`"1-0-Blue"`	The score and state of the game are unchanged since the last call to `getScore`.
`game.recordPlay(3);`		Team 2 earns 3 points.
`info = game.getScore();`	`"1-3-Blue"`	
`game.recordPlay(1);`		Team 2 earns 1 point.
`game.recordPlay(0);`		Team 2's play failed, so team 1 is now active.
`info = game.getScore();`	`"1-4-Red"`	
`game.recordPlay(0);`		Team 1's play failed, so team 2 is now active.
`game.recordPlay(4);`		Team 2 earns 4 points.
`game.recordPlay(0);`		Team 2's play failed, so team 1 is now active.
`info = game.getScore();`	`"1-8-Red"`	

FINAL Unit3 FRQ - Topic order

Statement	Value Returned (blank if none)	Explanation
`Scoreboard match = new Scoreboard("Lions", "Tigers");`		`match` is a new and independent `Scoreboard` object.
`info = match.getScore();`	`"0-0-Lions"`	
`info = game.getScore();`	`"1-8-Red"`	

Write the complete `Scoreboard` class. Your implementation must meet all specifications and conform to the examples shown in the preceding table.

10. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the implementation of the `AddtionPattern` class, which generates a number pattern. The `AdditionPattern` object is constructed with two positive integer parameters, as described below.

The first positive integer parameter indicates the starting number in the pattern.
The second positive integer parameter indicates the value that is to be added to obtain each subsequent number in the pattern.

The `AdditionPattern` class also supports the following methods.

`currentNumber`, which returns the current number in the pattern
`next`, which moves to the next number in the pattern
`prev`, which moves to the previous number in the pattern or takes no action if there is no previous number

The following table illustrates the behavior of an `AdditionPattern` object that is instantiated by the following statement.

```
AdditionPattern plus3 = new AdditionPattern(2, 3);
```

Method Call	Value Returned (blank if no value)	Explanation
`plus3.currentNumber();`	`2`	The current number is initially the starting number in the pattern.
`plus3.next();`		The pattern adds `3` each time, so move to the next number in the pattern (`5`).
`plus3.currentNumber();`	`5`	The current number is `5`.
`plus3.next();`		The pattern adds `3` each time, so move to the next number in the pattern (`8`).
`plus3.next();`		The pattern adds `3` each time, so move to the next number in the pattern (`11`).

FINAL Unit3 FRQ - Topic order

`plus3.currentNumber();`	`11`	The current number is `11`.
`plus3.prev();`		Move to the previous number in the pattern (`8`).
`plus3.prev();`		Move to the previous number in the pattern (`5`).
`plus3.prev();`		Move to the previous number in the pattern (`2`).
`plus3.currentNumber();`	`2`	The current number is `2`.
`plus3.prev();`		There is no previous number in the pattern prior to `2`, so no action is taken.
`plus3.currentNumber();`	`2`	The current number is `2`.

Write the complete `AditonPattern` class. Your implementation must meet all specifications and conform to all examples.